

- 聊天室项目TIO框架详解
 - 概述
 - 1. TIO框架架构概览
 - 1.1 核心组件结构
 - 2. TIO配置管理
 - 2.1 TIO服务配置类
 - 2.2 配置参数说明
 - 3. TIO服务启动机制
 - 3.1 WebSocket服务启动器
 - 3.2 服务启动流程
 - 3.3 服务器配置设置
 - 3.4 服务启动完成
 - 4. TIO消息处理机制
 - 4.1 WebSocket消息处理器
 - 4.2 WebSocket握手处理
 - 5. TIO连接生命周期管理
 - 5.1 服务器监听器
 - 6. TIO工具类及应用
 - 6.1 用户在线状态管理
 - 6.2 用户离线状态判断
 - 6.3 用户属性管理
 - 6.4 聊天室ID生成工具
 - 6.5 群聊室ID生成
 - 7. TIO框架在项目中的应用特点
 - 7.1 核心优势
 - 7.2 项目集成特色
 - 7.3 技术架构层次
 - 8. 关键配置参数汇总
 - 9. TIO框架的消息流转过程
 - 9.1 完整的消息处理流程
 - 9.2 连接管理机制
 - 总结

聊天室项目TIO框架详解

概述

TIO (t-io) 是一个基于Java的网络通信框架，本项目使用TIO的WebSocket模块实现实时通信功能。TIO框架在本聊天室项目中承担着核心的网络通信职责，负责WebSocket连接管理、消息处理、心跳检测等关键功能。

1. TIO框架架构概览

1.1 核心组件结构

本项目中TIO框架的核心组件包括：

- 配置类 (JRTioConfig)：TIO服务器的配置管理
- 启动器 (JRWebSocketStarter)：WebSocket服务的启动和初始化
- 消息处理器 (JRWsMsgHandler)：WebSocket消息的业务处理
- 监听器 (JRWsTioServerListener)：连接生命周期事件监听
- 工具类 (TioUtil)：TIO相关的实用方法集合

2. TIO配置管理

2.1 TIO服务配置类

文件位置：`c:\code\project\chatroom\dot-chat-server\src\main\java\com\dot\msg\chat\tio\config\JRTioConfig.java`

```
// 第18-45行
@Data
@Component
@ConfigurationProperties(prefix = "tio.server")
public class JRTioConfig {

    /**
     * 监听端口
     */
    private int port = 9326;

    /**
     * 监听的ip null表示监听所有，并不指定ip
     */
}
```

```
private String serverIp = null;

/**
 * 协议名字(可以随便取，主要用于开发人员辨识)
 */
private String protocolName = "dot";

private String charset = "utf-8";

/**
 * 心跳超时时间，单位：毫秒
 */
private int heartbeatTimeout = 1000 * 60;
```

2.2 配置参数说明

配置项	默认值	说明
port	9326	WebSocket服务监听端口
serverIp	null	监听IP，null表示监听所有IP
protocolName	"dot"	协议名称，用于开发辨识
charset	"utf-8"	字符编码格式
heartbeatTimeout	60000	心跳超时时间（毫秒）

3. TIO服务启动机制

3.1 WebSocket服务启动器

文件位置：`c:\code\project\chatroom\dot-chat-server\src\main\java\com\dot\msg\chat\tio\start\JRWebsocketStarter.java`

```
// 第17-30行
@Slf4j
@Component
public class JRWebsocketStarter {

    @Resource
    private JRWsMsgHandler wsMsgHandler;
```

```
@Resource
private JRipStatListener ipStatListener;

@Resource
private JRWsTioServerListener wsTioServerListener;

@Resource
private JRTioConfig jrTioConfig;
```

3.2 服务启动流程

服务启动方法使用 `@PostConstruct` 注解，在Spring容器初始化时自动执行：

文件位置： `c:\code\project\chatroom\dot-chat-server\src\main\java\com\dot\msg\chat\tio\start\JRWebsocketStarter.java`

```
// 第41-70行
@PostConstruct
public void start() throws Exception {
    log.info("🚀 === WebSocket服务启动开始 ===");

    try {
        // 配置检查
        log.debug("📄 检查TIO配置:");
        log.debug("    端口: {}", jrTioConfig.getPort());
        log.debug("    协议名称: {}", jrTioConfig.getProtocolName());
        log.debug("    心跳超时: {}ms", jrTioConfig.getHeartbeatTimeout());
        log.debug("    字符集: {}", jrTioConfig.getCharset());
        log.debug("    服务器IP: {}", jrTioConfig.getServerIp());

        // 创建WebSocket服务器
        log.info("🔧 创建WebSocket服务器, 端口: {}", jrTioConfig.getPort());
        WsServerStarter wsServerStarter = new
WsServerStarter(jrTioConfig.getPort(), wsMsgHandler);

        // 获取服务器配置
        log.debug("⚙️ 配置TIO服务器参数");
        TioServerConfig serverTioConfig = wsServerStarter.getTioServerConfig();
        serverTioConfig.setName(jrTioConfig.getProtocolName());
```

3.3 服务器配置设置

文件位置： `c:\code\project\chatroom\dot-chat-server\src\main\java\com\dot\msg\chat\tio\start\JRWebsocketStarter.java`

```
// 第71-89行
serverTioConfig.setTioServerListener(wsTioServerListener);
log.debug("    设置服务器监听器: ✅");

// 设置ip监控
serverTioConfig.setIpStatListener(ipStatListener);
log.debug("    设置IP统计监听器: ✅");

// 设置ip统计时间段
serverTioConfig.ipStats.addDurations(jrTioConfig.IPSTAT_DURATIONS);
log.debug("    设置IP统计时间段: {} 个时间段", jrTioConfig.IPSTAT_DURATIONS.length);

// 设置心跳超时时间
serverTioConfig.setHeartbeatTimeout(jrTioConfig.getHeartbeatTimeout());
log.debug("    设置心跳超时时间: {}ms", jrTioConfig.getHeartbeatTimeout());
```

3.4 服务启动完成

文件位置: c:\code\project\chatroom\dot-chat-server\src\main\java\com\dot\msg\chat\tio\start\JRWebsocketStarter.java

```
// 第104-113行
// 启动服务器
log.info("🚀 启动WebSocket服务器...");
wsServerStarter.start();
log.info("✅ === WebSocket服务启动成功 ===");
log.info("🌐 WebSocket服务器运行在: ws://{}:{}/websocket/",
    jrTioConfig.getServerIp() != null ? jrTioConfig.getServerIp() : "0.0.0.0",
    jrTioConfig.getPort());
```

4. TIO消息处理机制

4.1 WebSocket消息处理器

文件位置: c:\code\project\chatroom\dot-chat-server\src\main\java\com\dot\msg\chat\tio\handler\JRWsMsgHandler.java

```
// 第45-75行
@Slf4j
@Component
public class JRWsMsgHandler implements IWsMsgHandler {

    @Resource
    private TokenManager tokenManager;

    @Resource
    private ChatUserService chatUserService;

    @Resource
    private ChatGroupMemberService chatGroupMemberService;

    @Resource
    private ChatRoomService chatRoomService;

    @Resource
    private ChatMsgService chatMsgService;

    @Resource
    private ChatMsgSendService chatMsgSendService;

    @Resource
    private NotifyMsgService notifyMsgService;

    @Resource
    private ApplicationEventPublisher eventPublisher;

    @Resource(name = "redisUtil")
    private RedisUtil redisUtil;
}
```

4.2 WebSocket握手处理

文件位置： `c:\code\project\chatroom\dot-chat-server\src\main\java\com\dot\msg\chat\tio\handler\JRWsMsgHandler.java`

```
// 第82-85行
/**
 * 握手时走这个方法，业务可以在这里获取cookie，request参数等
 */
@Override
public HttpResponse handshake(HttpRequest request, HttpResponse httpResponse,
ChannelContext channelContext) {
}
```

5. TIO连接生命周期管理

5.1 服务器监听器

文件位置： `c:\code\project\chatroom\dot-chat-server\src\main\java\com\dot\msg\chat\tio\listener\JRWsTioServerListener.java`

```
// 第25-35行
/**
 * 用户根据情况来完成该类的实现
 */
@Slf4j
@Component
public class JRWsTioServerListener extends WsTioServerListener {
```

该监听器继承了TIO框架的 `WsTioServerListener` 类，负责处理连接建立、连接断开等生命周期事件。

6. TIO工具类及应用

6.1 用户在线状态管理

文件位置： `c:\code\project\chatroom\dot-chat-server\src\main\java\com\dot\msg\chat\tio\util\TioUtil.java`

```
// 第32-41行
/**
 * 判断用户是否在线
 *
 * @param tioConfig 服务器Tio配置
 * @param userId 用户ID
 * @return 是否在线
 */
public static boolean isOnline(TioConfig tioConfig, String userId) {
    boolean isOnline = true;
    SetWithLock<ChannelContext> contextSetWithLock = Tio.getByUserid(tioConfig,
userId);
    if (contextSetWithLock == null) {
        isOnline = false;
        log.warn("接收用户处于离线状态,toUserId:{})", userId);
    }
    return isOnline;
}
```

6.2 用户离线状态判断

文件位置： `c:\code\project\chatroom\dot-chat-server\src\main\java\com\dot\msg\chat\tio\util\TioUtil.java`

```
// 第43-50行
/**
 * 判断用户是否离线
 *
 * @param tioConfig 服务器Tio配置
 * @param userId    用户ID
 * @return 是否离线
 */
public static boolean isOffline(TioConfig tioConfig, String userId) {
    return !isOnline(tioConfig, userId);
}
```

6.3 用户属性管理

文件位置： `c:\code\project\chatroom\dot-chat-server\src\main\java\com\dot\msg\chat\tio\util\TioUtil.java`

```
// 第125-135行
/**
 * 设置属性值
 *
 * @param tioConfig 服务器Tio配置
 * @param userId    用户ID
 * @param key       属性名
 * @param value     值
 */
public static void setAttribute(TioConfig tioConfig, String userId, String key,
Object value) {
    SetWithLock<ChannelContext> channelContexts = Tio.getByUserId(tioConfig,
userId);
    if (channelContexts == null) {
        log.error("用户处于离线状态,userId:{", userId);
        return;
    }
    channelContexts.getObj().stream().findFirst().ifPresent(channelContext -> {
        channelContext.setAttribute(key, value);
    });
}
```


6.4 聊天室ID生成工具

文件位置： `c:\code\project\chatroom\dot-chat-server\src\main\java\com\dot\msg\chat\tio\util\TioUtil.java`

```
// 第195-205行
/**
 * 获取聊天室ID
 *
 * @param sendUserId 发送用户ID
 * @param toUserId 接收用户ID
 * @return 聊天室ID
 */
public static String generateChatId(Integer sendUserId, Integer toUserId) {
    List<Integer> userIds = new ArrayList<>();
    userIds.add(sendUserId);
    userIds.add(toUserId);
    Collections.sort(userIds);
    return CollUtil.join(userIds, "_");
}
```

6.5 群聊室ID生成

文件位置： `c:\code\project\chatroom\dot-chat-server\src\main\java\com\dot\msg\chat\tio\util\TioUtil.java`

```
// 第217-223行
/**
 * 获取聊天室ID
 *
 * @param groupId 群ID
 * @return 聊天室ID
 */
public static String generateChatId(Integer groupId) {
    return "G_" + groupId;
}
```

7. TIO框架在项目中的应用特点

7.1 核心优势

- 1. **高性能**：基于NIO技术，支持大量并发连接
- 2. **简单易用**：提供简洁的API，降低网络编程复杂度
- 3. **功能完善**：内置心跳检测、IP监控、SSL支持等功能
- 4. **生态丰富**：支持HTTP、WebSocket、TCP等多种协议

7.2 项目集成特色

- 1. **Spring集成**：通过注解和依赖注入无缝集成Spring框架
- 2. **配置管理**：使用Spring的配置属性绑定，便于环境切换
- 3. **监控统计**：内置IP访问统计和连接监控功能
- 4. **事件驱动**：通过监听器模式处理连接生命周期事件

7.3 技术架构层次



8. 关键配置参数汇总

组件	配置项	位置	说明
服务端 口	port=9326	JRTioConfig.java:29 行	WebSocket服务监听 端口
协议名 称	protocolName="dot"	JRTioConfig.java:35 行	TIO协议标识名称
心跳超 时	heartbeatTimeout=60000	JRTioConfig.java:41 行	心跳超时时间（毫 秒）

组件	配置项	位置	说明
字符编码	charset="utf-8"	JRTioConfig.java:37行	消息编码格式
服务器IP	serverIp=null	JRTioConfig.java:26行	监听IP地址

9. TIO框架的消息流转过程

9.1 完整的消息处理流程

1. **连接建立**：客户端发起WebSocket连接，TIO框架接收连接请求
2. **握手处理**：`JRWsMsgHandler.handshake()`处理WebSocket握手
3. **消息接收**：TIO框架接收客户端消息
4. **消息解析**：根据消息类型路由到对应的处理方法
5. **业务处理**：执行具体的业务逻辑
6. **消息发送**：通过TIO的API发送响应消息给客户端

9.2 连接管理机制

TIO框架通过 `ChannelContext` 对象管理每个WebSocket连接，支持：

- 用户ID绑定：将连接与具体用户关联
- 属性存储：在连接上下文中存储用户状态
- 群组管理：支持将连接加入特定的群组
- 广播消息：向群组内的所有连接发送消息

总结

TIO框架在本聊天室项目中扮演着网络通信的核心角色，通过其高性能的NIO架构、完善的功能特性和简洁的API设计，为实时聊天应用提供了稳定可靠的技术基础。项目通过合理的架构设计，将TIO框架与Spring Boot无缝集成，实现了高并发、低延迟的实时通信能力，为用户提供了流畅的聊天体验。